

UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Avance 20/10/2026 - CodeMichi

Presentado a:

Grupos de Investigación Multimedia Interactiva

Presentado por:

Yader Ibraldo Quiroga Torres – 20222020034

Jhonathan De La Torre - 20222020033

Facultad de Ingeniería

Ingeniería de sistemas

20 de Octubre 2026

REPORTE DE AVANCES — MICHICODE 2.0

Proyecto: Juego educativo de programación por bloques (Gato minero 2.5D)

1. RESUMEN EJECUTIVO

Durante esta fase de desarrollo hemos completado la transición de lo que era un prototipo funcional básico a una arquitectura de juego robusta, estable y fácil de escalar, la prioridad principal fue desacoplar la lógica de los niveles del código base para evitar tener que duplicar escenas o reprogramar eventos cada vez que queramos diseñar un mapa nuevo.

Como resultado, logramos implementar un sistema en el que cada nivel se define completamente por datos, además, construimos una capa visual y de retroalimentación mucho más clara para el estudiante: se agregaron cinemáticas de entrada y salida, diálogos contextuales según la acción del jugador, un sistema de previsualización de ruta en tiempo real y una interfaz de usuario completamente renovada estilo "cristal neón".

Estado de la entrega: El Nivel 1 (tutorial) ya es totalmente jugable de principio a fin, el flujo completo abarca desde la cinemática e instrucciones iniciales, pasando por la construcción del programa con asistencia visual en el mapa, hasta la recolección de minerales, manejo de errores explicativos, cinemática de victoria y transición fluida hacia el siguiente nivel.

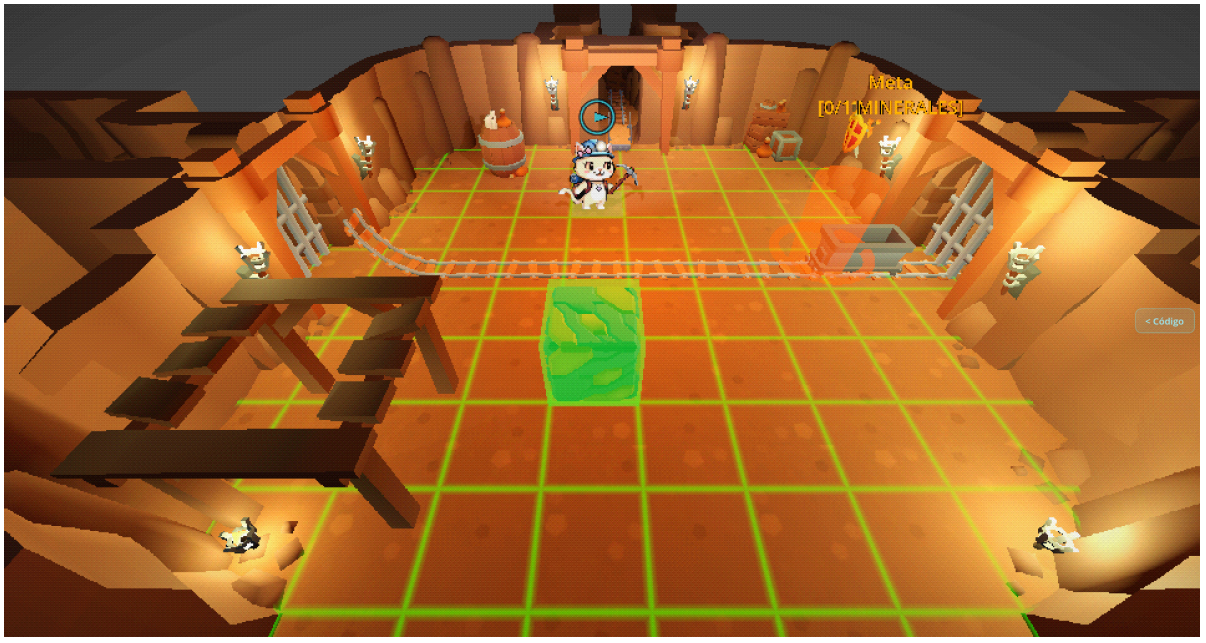
2. DETALLE DE IMPLEMENTACIÓN POR SISTEMA

2.1 Jugador y Sistema de Movimiento (`GridPlayer.gd`)

El control del personaje requirió varias capas de ajuste para garantizar que la ejecución por bloques se sienta precisa sobre la grilla 3D, pero sin perder la fluidez de un juego convencional.

- **API de comandos y ejecución:** Mantenemos la interfaz estándar `execute_command()` para procesar las instrucciones `avanzar`, `girar_izq`, `girar_der` y `activar` / `picar`. Esto garantiza compatibilidad directa con el motor de ejecución secuencial.
- **Desplazamiento discreto sobre el mapa:** El personaje se desplaza en celdas exactas de 2 metros. Para evitar movimientos rígidos, implementamos interpolaciones suaves (Sine/EaseOut) conectadas directamente con el `AnimationTree`, coordinando las animaciones de caminar, reposo y minado.

- **Manejo de orientación y cámara:** El sistema de direcciones maneja los cuatro puntos cardinales (N, E, S, O) y los traduce al espacio 2D del sprite del personaje, esto permite que la gata apunte hacia la dirección correcta independientemente del ángulo desde el que la cámara esté mirando el mapa.
- **Colisiones y físicas:** Usamos `test_move` para detectar muros y rocas antes de realizar el movimiento. Si el personaje intenta avanzar contra un obstáculo, el movimiento se interrumpe, se ejecuta un pequeño rebote visual ("bump") y se envía un registro del choque. Las gemas y la meta no bloquean el paso físico.
- **Lógica de minado y Checkpoints:** Al ejecutar la acción de picar sobre una celda con mineral, se dispara la animación, se generan partículas de impacto y el mineral se remueve del registro del mapa, un cambio importante en esta versión es que **cada mineral picado con éxito guarda automáticamente un checkpoint**, evitando que el usuario tenga que rehacer partes del nivel que ya resolvió bien.
- **Simulación interna de trayectoria:** Diseñamos una función dedicada (`simulate_program_path`) que toma la lista actual de comandos y "calcula" la ruta que recorrería el personaje sin moverlo realmente, esta simulación es la que alimenta la previsualización neón en el suelo.
- **Indicador de orientación en pantalla:** Colocamos un marcador flotante (flecha y anillo neón) sobre el personaje, como la cámara es 2.5D, este indicador proyecta la dirección hacia la pantalla para que el jugador nunca se confunda de sentido al colocar giros a la izquierda o derecha.
- **Modo de prueba libre (Debug):** Implementamos un modo de navegación con teclado (tecla T) para mover al personaje libremente por el mapa durante las pruebas de desarrollo permitiéndonos verificar zonas de choque y celdas rápidamente.



2.2 Ejecutor de Instrucciones (`CommandRunner.gd`)

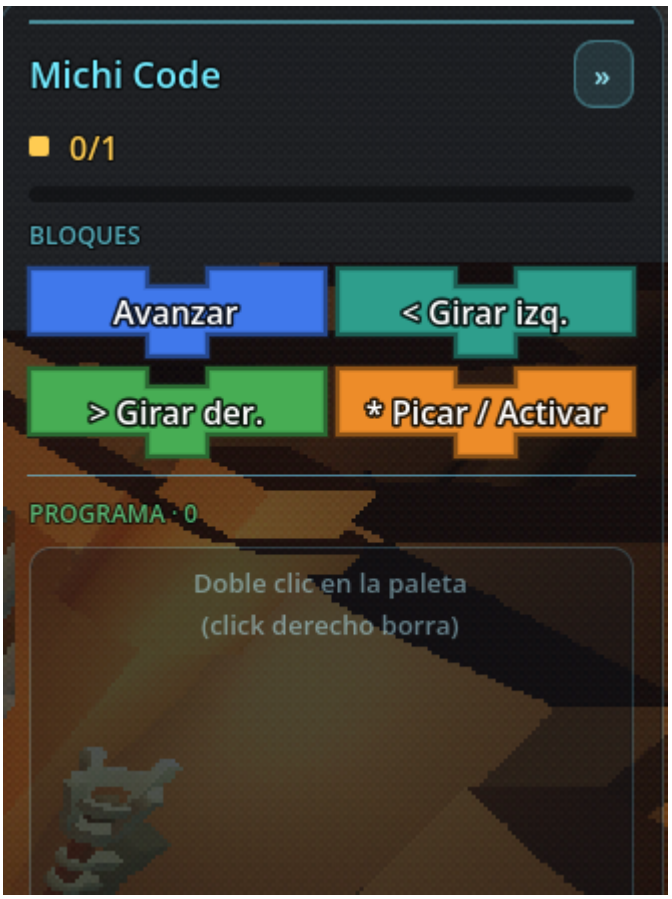
Es el motor encargado de tomar la lista de bloques que el usuario colocó en el área de trabajo y procesarlos uno por uno.

- Mantiene la secuencia lineal enviando avisos cuando inicia un paso, cuando se completa o cuando la rutina termina (ya sea por éxito o por fallo).
- **Control de pausas:** Agregamos la posibilidad de pausar y reanudar la secuencia, esto es fundamental cuando el personaje interactúa con elementos que disparan cuadros de diálogo, evitando que la animación o el movimiento continúen mientras el jugador lee el texto.

2.3 Bloques Visuales y Espacio de Trabajo (`Block.gd`, `Workspace.gd`)

La interfaz de bloques pasó por un rediseño importante para que sea más intuitiva y ofrezca mejor retroalimentación al estudiante mientras programa.

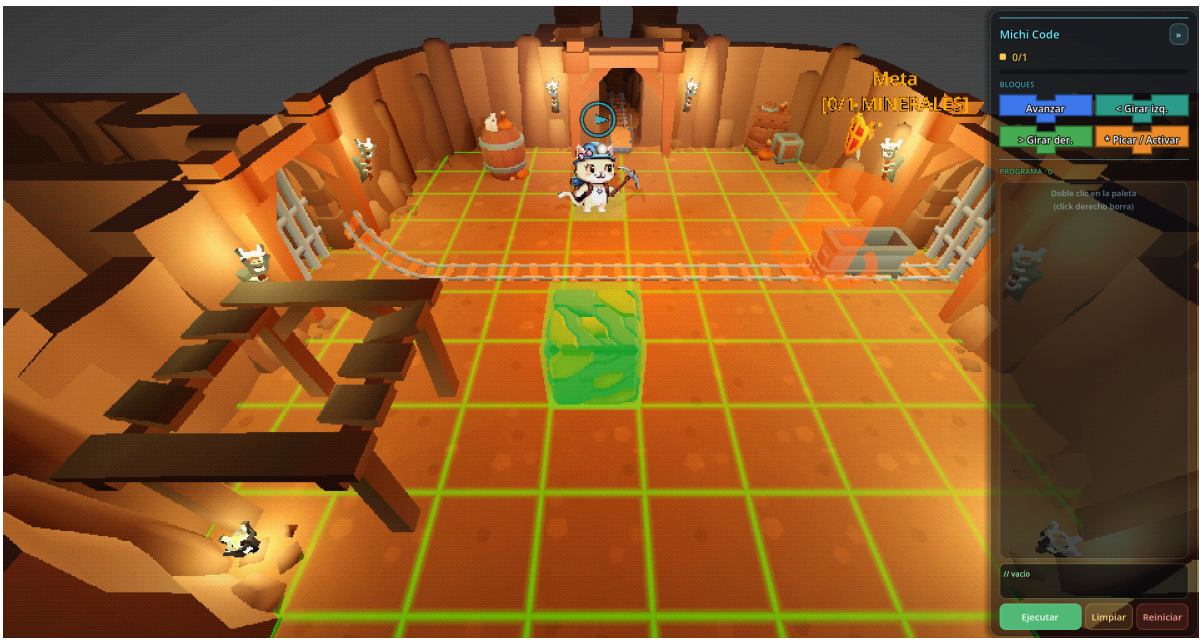
- **Conectores estilo rompecabezas:** Los bloques ahora se dibujan dinámicamente con muescas superiores y pestañas inferiores para reforzar visualmente la idea de secuencia encadenada.
- **Código de colores por categoría:** Clasificamos las acciones por color: azul para desplazamiento recto, verde-azul para giros a la izquierda, verde para giros a la derecha y naranja para acciones del entorno (picar/activar).
- **Estados de ejecución:** Mientras la secuencia corre, cada bloque cambia su estado en tiempo real: amarillo cuando se está ejecutando, verde cuando la acción fue exitosa y rojo con un destello si el paso provocó un choque o un fallo de lógica.
- **Interacciones rápidas:** Además de arrastrar y soltar, añadimos el doble clic en la paleta para agregar bloques rápidamente al final de la lista, y clic derecho sobre cualquier bloque del área de trabajo para eliminarlo al instante.
- **Espacio de trabajo reactivo:** El área de trabajo notifica cualquier cambio en la estructura de bloques de forma inmediata, lo que permite refrescar el código simplificado y la ruta proyectada en el piso en tiempo real.



2.4 Interfaz de Usuario y HUD (`CodeUI.gd`)

Reestructuramos el panel lateral de programación para equilibrar la visibilidad de las herramientas con el espacio del escenario en 3D.

- **Panel lateral "Cristal Neón":** La estructura del panel utiliza un fondo translúcido con bordes iluminados, ejsto permite al jugador seguir viendo el mapa y la ruta del personaje por debajo de la interfaz sin que el panel tape todo el escenario.
- **Pestaña colapsable:** El panel de código se puede ocultar por completo hacia un lateral (o con la tecla C), lo que resulta muy útil para revisar el mapa antes de armar la estrategia.
- **Progreso del nivel:** Incluye el título del mapa actual, el recuento de minerales recolectados con una barra de estado visual y una vista simplificada en pseudo-código numerado para conectar la lectura de los bloques con sintaxis de texto simple.
- **Controles y atajos:** Botones claros con colores según su acción (Ejecutar en verde, Limpiar en dorado, Detener/Reiniciar en rojo) que además responden a atajos de teclado estándar (F5 para correr el código, R para reiniciar).



2.5 Previsualización de Ruta en Tiempo Real (`PathPreview.gd`)

Una de las adiciones más importantes para la usabilidad del juego es el sistema de proyección de ruta sobre el terreno.

- **Rastro neón sobre el suelo:** A medida que el usuario encadena bloques en el área de trabajo, el mapa dibuja un camino iluminado que muestra exactamente la trayectoria que seguirá el personaje.
- **Flechas de dirección:** Cada casilla proyectada incluye un marcador que indica los giros que hará el personaje en ese punto exacto, ayudando a anticipar si un giro quedó al revés antes de ejecutar.
- **Detección visual de choques:** Si la secuencia de bloques hace que el personaje choque contra un muro o intente picar donde no hay nada, el rastro cambia de color a rojo en la casilla del impacto, marcando el punto exacto donde fallará el código.
- **Renderizado por capas inteligente:** Ajustamos las capas del shader para que, si el personaje camina sobre o por delante de la ruta proyectada, el rastro siga siendo visible a través de los elementos del mapa.



2.6 Arquitectura de Niveles y Gestión de Estado

Para lograr que el proyecto sea escalable a decenas de niveles sin duplicar trabajo técnico, dividimos la lógica en varios componentes especializados.

- **Niveles basados en datos (`LevelData.gd`):** Creamos un tipo de recurso donde definimos toda la configuración de un mapa en un solo archivo, aquí se especifica la escena 3D a usar, posición inicial y orientación del personaje, ubicación de minerales, casilla de la meta, cámara por defecto y los bloques de programación disponibles para ese nivel en específico.
- **Administrador del Juego (`GameManager.gd`):** Funciona como el cerebro central, se encarga de instanciar los niveles a partir de sus archivos de datos, gestionar la lógica de victoria/derrota, inyectar datos en los diálogos dinámicamente y coordinar la carga del siguiente mapa de la lista.
- **Traductor de errores a lenguaje amigable:** Si el programa falla (por ejemplo, si el personaje choca o intenta ir a la meta sin terminar la recolección) el sistema traduce el error técnico a un mensaje pedagógico que se muestra en un diálogo para orientar al estudiante.
- **Controlador de Cámara (`CameraController.gd`):** Maneja los encuadres del juego, soporta seguimiento suave del jugador, giros para enfocar objetivos específicos (como la puerta abriéndose) y efectos de sacudida leve cuando ocurre una colisión.

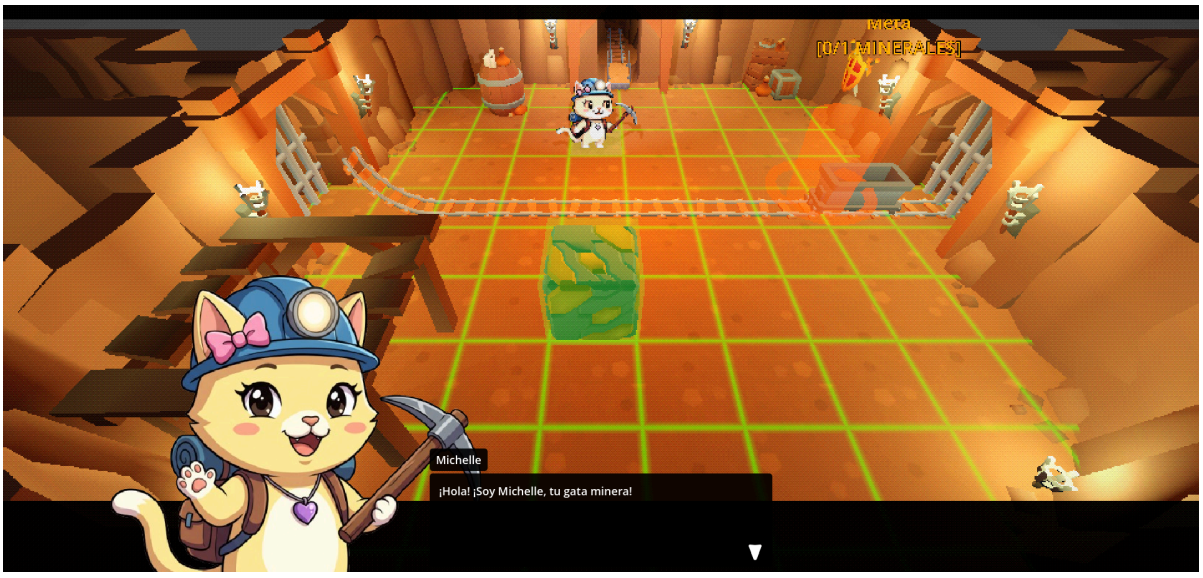
2.7 Entidades del Escenario (`Mineral.gd`, `Goal.gd`)

- **Minerales:** Cuentan con un efecto de pulso neón constante para destacar dentro de la grilla 3D. Al ser minados, activan sistemas de partículas con fragmentos

dorados, notifican al contador central de la UI y desactivan su presencia en el mapa.
- **Meta y Puerta de Salida:** Mantiene un estado bloqueado (iluminado en naranja) mientras existan minerales pendientes en el escenario, una vez recolectado el último mineral, la meta se activa en verde emitiendo un haz de luz y mostrando la etiqueta de habilitada, al pisarla, la puerta de salida ejecuta su animación de apertura y el nivel se da por superado.

2.8 Sistema de Cinemáticas y Diálogos

- **Secuencias de entrada y salida:** Diseñamos un sistema de corte cinematográfico que aplica barras negras ajustadas (letterbox), oculta temporalmente la interfaz de bloques y mueve la cámara para presentar el mapa o mostrar la animación de victoria antes de avanzar de nivel.
- **Integración con Dialogic:** Implementamos secuencias de diálogo para dar la bienvenida al jugador, celebrar la recolección de minerales y explicar la razón exacta de un fallo cuando la rutina de bloques no logra el objetivo.



3. PRÓXIMOS PASOS EN EL DESARROLLO

1. **Diseño del Nivel 3:** Crear el archivo de datos correspondiente, armar la disposición del mapa en el editor y configurar la secuencia de diálogos introductorios.
2. **Menú principal y guardado:** Implementar una pantalla de selección de mapa que registra el progreso del usuario y los niveles que ya han sido superados.
3. **Nuevas estructuras de programación:** Comenzar la investigación e integración de bloques más avanzados, comenzando por bucles simples (`repetir N veces`) y condicionales sencillos (`si hay mineral adelante`).

4. **Capa de audio:** Integrar efectos de sonido para los pasos del personaje, la acción de picar, el acople de bloques en la interfaz y música ambiental adaptada a cada nivel.

